

Paradigmas de Programação

Programação Lógica

Parte A – Lógica Proposicional Booleana

1. Julgue a afirmativa a seguir como **verdadeira** ou **falsa**: “Afirmar que uma proposição p é, ao mesmo tempo, verdadeira e falsa, violaria o Princípio do Terceiro Excluído”.

Solução:

Falsa. A afirmativa viola o **Princípio da Não-Contradição**: uma proposição não pode ser, ao mesmo tempo, verdadeira e falsa. O **Princípio do Terceiro Excluído** afirma que toda proposição ou é verdadeira ou é falsa, sem uma terceira possibilidade.

2. Na canção Maricotinha, Dorival Caymmi afirma que “Se fizer bom tempo amanhã eu vou”. Assumindo que esta proposição é verdadeira, assinale a única situação impossível:

- ☐ Fez bom tempo e ele foi
- ☐ Fez bom tempo e ele não foi
- ☐ Não fez bom tempo e ele foi
- ☐ Não fez bom tempo e ele não foi

Solução:

Letra B: “Fez bom tempo e ele não foi”. A condicional $p \rightarrow q$ (“Se fizer bom tempo, eu vou”) só é falsa quando o antecedente p é verdadeiro e o consequente q é falso. As demais combinações tornam a proposição verdadeira.

3. Qual é o menor número natural x que torna a sentença aberta “ $x/2 > \pi$ ” verdadeira?

Solução:

$$\frac{x}{2} > \pi \iff x > 2\pi.$$

Como $2\pi \approx 6,28$, o menor número natural que torna a sentença verdadeira é $x = 7$.

Parte B – Fundamentos da programação lógica

4. Considere o código Prolog abaixo:

```
1 time('5am, wood again', 2, 'mg').
2 time('Amigos do Mortandela', 3, 'mg').
3 time('C++ ou uma linguagem misteriosa?', 2, 'go').
4 time('Lone Wolves', 2, 'df').
5 time('Monkeys', 1, 'go').
6 time('Teorema de Offson', 1, 'df').
```

```

7 time('Teorema do Chinês Viajante', 3, 'df').
8 time('Torcida Pão de Alho', 1, 'mg').
9
10 campeão(X) :- time(X, 1, _).
11
12 selecionado(X) :- campeão(X).
13 selecionado(X) :- time(X, _, 'mg').

```

- Identifique os predicados (nome e aridade).
- Determine o número de fatos, regras e cláusulas.
- Observe a saída da consulta

```
?- selecionado(X).
```

Proponha uma correção para o código para que a saída da consulta liste corretamente os selecionados.

Solução:

- Predicados: `time`/3 (fato), `campeao`/1 (regra) e `selecionado`/1 (regra).
- Há 8 fatos (`time`/3), 3 regras (uma de `campeao`/1 e duas de `selecionado`/1) e, portanto, 11 cláusulas no total.
- A consulta retorna as equipes campeãs (`time`/3 com 1 no segundo argumento) e, em seguida, as equipes do grupo 'mg': 'Monkeys', 'Teorema de Offson', 'Torcida Pão de Alho', '5am, wood again', 'Amigos do Mortandela' e novamente 'Torcida Pão de Alho'. A equipe 'Torcida Pão de Alho' aparece duplicada porque é, ao mesmo tempo, campeã e do grupo 'mg'. Para eliminar a duplicação, basta impedir que a segunda regra liste as campeãs:

```

selecionado(X) :- campeão(X).
selecionado(X) :- time(X, _, 'mg'), \+ campeão(X).

```

5. Considere os seguintes fatos:

```

1 p(a, b).
2 p(a, c).
3 p(b).
4
5 g(a).
6 g(b).
7 g(a, b).

```

Determine o resultado das seguintes consultas. Caso o resultado seja verdadeiro, indique também o valor das variáveis lógicas, quando for o caso.

- ?- `p(b, a)`.
- ?- `p(a, b)`.
- ?- `g(X)`.
- ?- `p(a, Y)`.

Solução:

- ?- `p(b, a)`. ⇒ **falso**: não há fato `p(b, a)` (o fato `p(b)` tem aridade 1).
- ?- `p(a, b)`. ⇒ **verdadeiro**: corresponde ao fato `p(a, b)`.
- ?- `g(X)`. ⇒ **verdadeiro**, com $X = a$ e $X = b$ (fatos `g(a)` e `g(b)`; o fato `g(a, b)` tem aridade 2 e não unifica).
- ?- `p(a, Y)`. ⇒ **verdadeiro**, com $Y = b$ e $Y = c$ (fatos `p(a, b)` e `p(a, c)`).

6. Considere uma consulta composta de Prolog e as quatro portas que controlam o fluxo da execução. Descreva o cenário no qual a saída da consulta exibirá as variáveis lógicas e seus respectivos valores. Explique quando é

possível inserir ponto-e-vírgula após o resultado de uma consulta e qual é o significado desta inserção, em termos do fluxo de controle.

Solução:

As quatro portas do fluxo de execução são **call** (chamada do predicado), **exit** (sucesso da cláusula), **redo** (tentativa de obter uma nova solução, por retrocesso) e **fail** (fracasso da cláusula).

A saída exibe as variáveis lógicas e seus valores quando a consulta **sai** pela porta **exit**: ou seja, quando uma cláusula é satisfeita e o Prolog mostra a substituição (as ligações) das variáveis que tornam a consulta verdadeira.

O ponto-e-vírgula (;) pode ser digitado sempre que o Prolog exibe uma solução e fica aguardando confirmação. Ao digitá-lo, o usuário aciona a porta **redo**: o Prolog desfaz as últimas ligações e tenta, por retrocesso (*backtracking*), encontrar a próxima solução. Quando não há mais soluções, a consulta termina em **fail** e o Prolog responde **false**.

7. Considere o código Prolog abaixo, onde o predicado **matricula**/2 indica que o estudante está matriculado na disciplina indicada e o predicado **estudante**/3 informa que o estudante do semestre indicado cursa a graduação dada no terceiro argumento:

```
1 estudante(ana, 3, 'Engenharia de Software').
2 estudante(beto, 1, 'Engenharias').
3 estudante(carlos, 4, 'Engenharia de Energia').
4 estudante(diane, 2, 'Engenharias').
5 estudante(euler, 1, 'Engenharias').
6 estudante(fabio, 5, 'Engenharia de Software').
7 estudante(gustavo, 8, 'Engenharia de Software').
8 estudante(heitor, 7, 'Engenharia de Energia').
9 estudante(ian, 3, 'Engenharias').
10
11 matricula('Cálculo 1', ana).
12 matricula('Cálculo 1', fabio).
13 matricula('Cálculo 1', diane).
14 matricula('Cálculo 1', euler).
15 matricula('Cálculo 1', gustavo).
16 matricula('Cálculo 1', ian).
17
18 matricula('IAL', beto).
19 matricula('IAL', diane).
20 matricula('IAL', euler).
21
22 matricula('APC', carlos).
23 matricula('APC', fabio).
24 matricula('APC', gustavo).
25 matricula('APC', ian).
```

- (a) Implemente a regra **tem_calouros**/1, que retorna as disciplinas que tem calouros matriculados.
- (b) Implemente a regra **turma_mista**/1, que retorna as disciplinas que tem ao menos um estudante de cada curso.
- (c) Implemente a regra **software**/0, que imprime todos os estudantes de Engenharia de Software, um estudante por linha.

Solução:

Um calouro é um estudante do semestre 1. Uma disciplina tem calouros quando algum estudante do semestre 1 está matriculado nela; uma turma é mista quando possui ao menos um estudante de cada um dos três cursos; e **software**/0 imprime os estudantes de Engenharia de Software usando um laço de falha (**fail**):

```
% (a) Disciplinas com calouros (estudantes do semestre 1) matriculados
tem_calouros(D) :-
    matricula(D, S),
    estudante(S, 1, _).

% (b) Disciplinas com ao menos um estudante de cada curso
turma_mista(D) :-
    matricula(D, S1), estudante(S1, _, 'Engenharia de Software'),
```

```

matricula(D, S2), estudante(S2, _, 'Engenharias'),
matricula(D, S3), estudante(S3, _, 'Engenharia de Energia').

% (c) Imprime os estudantes de Engenharia de Software, um por linha
software :-
    estudante(S, _, 'Engenharia de Software'),
    write(S), nl,
    fail.

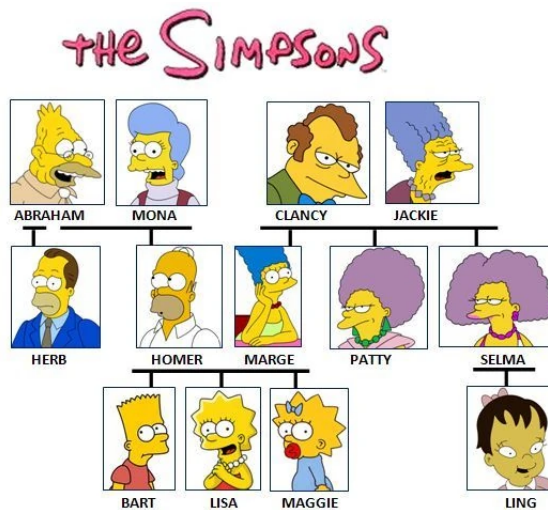
% Serve para que a consulta não retorne falso.
software.

    tem_calouros/1 retorna 'Cálculo 1' e 'IAL'; turma_mista/1 retorna 'APC'; e software/0 imprime ana,
fabio e gustavo.

```

Parte C – Regras

8. Considere a figura abaixo, que apresenta a árvore genealógica dos Simpsons:



Implemente os predicados abaixo, declarando a seguir os fatos relativos à família Simpson relacionados a cada predicado.

- (a) `male/1` e `female/1`, que indicam os homens e mulheres, respectivamente.
- (b) `father/2`, que indica o pai do respectivo membro da família.
- (c) `mother/2`, que indica a mãe do respectivo membro da família.

Implemente as regras abaixo e valide sua implementação com consultas na base de fatos declarados.

- (a) `uncle/2`, que indica o tio do respectivo membro da família.
- (b) `grandmother/2`, que indica a avó do respectivo membro da família.

Solução:

```

% Homens e mulheres
male(abraham). male(clancy). male(herb). male(homer). male(bart).
female(mona). female(jackie). female(marge). female(patty).
female(selma). female(lisa). female(maggie). female(ling).

% Pai
father(herb, abraham).
father(homer, abraham).
father(marge, clancy).
father(patty, clancy).
father(selma, clancy).
father(bart, homer).
father(lisa, homer).
father(maggie, homer).

```

```

% Mãe
mother(herb, mona).
mother(homer, mona).
mother(marge, jackie).
mother(patty, jackie).
mother(selma, jackie).
mother(bart, marge).
mother(lisa, marge).
mother(maggie, marge).
mother(ling, selma).

% Auxiliares
parent(P, X) :- father(P, X).
parent(P, X) :- mother(P, X).

sibling(A, B) :-
    father(A, F), father(B, F),
    mother(A, M), mother(B, M),
    A \= B.

% Tio: homem que é irmão de um dos pais
uncle(U, X) :-
    male(U),
    parent(P, X),
    sibling(U, P).

% Avó: mãe de um dos pais
grandmother(G, X) :-
    mother(G, P),
    parent(P, X).

```

A regra `uncle/2` usa o auxiliar `sibling/2` (irmãos compartilham o mesmo pai e a mesma mãe): o tio de X é um homem que é irmão de um dos pais de X . A regra `grandmother/2` diz que a avó de X é a mãe de um dos pais de X .

9. Implemente o predicado `distance/5`, que compute a distância percorrida S , em metros, em T segundos, a partir de um ponto inicial que fica a I metros da origem, com velocidade de V m/s e aceleração igual a A m/s².

Solução:

Usando a equação horária do movimento uniformemente variado,

$$S = I + V \cdot T + \frac{A \cdot T^2}{2},$$

o predicado (com assinatura `distance(I, V, T, A, S)`) é:

```

distance(I, V, T, A, S) :-
    S is I + V*T + A*T*T/2.

```

10. Implemente o predicado `divisores/2`, que computa o número de divisores positivos X de um inteiro positivo N . Utilize o predicado `mod/2`, que retorna o resto da divisão de X por Y . Em seguida, transforme este predicado em uma função aritmética e também em um operador infixado, sem associatividade, de modo que as consultas abaixo sejam possíveis e retornem os resultados indicados:

```

1 ?- 60 divisores X.
2 X = 12.
3
4 ?- R is divisores(20).
5 R = 6.

```

Solução:

O predicado conta, de N até 1, quantos valores dividem N , usando $\text{mod}/2$ e um acumulador:

```
:- use_module(library(arithmetic)).
:- arithmetic_function(divisores/1).
:- op(500, xfx, divisores).

divisores(N, X) :- divisores(N, N, 0, X).

divisores(N, D, Acc, X) :-
    (D > 0
    -> ((mod(N, D) =:= 0 -> succ(Acc, NewAcc) ; NewAcc = Acc),
        succ(NewD, D),
        divisores(N, NewD, NewAcc, X))
    ; X = Acc).
```

A diretiva `:- arithmetic_function(divisores/1).` registra `divisores/1` como função aritmética, permitindo `R is divisores(20)` ($R = 6$). A diretiva `:- op(500, xfx, divisores).` declara `divisores` como operador infixado sem associatividade, permitindo `?- 60 divisores X.` ($X = 12$).

Parte D - Listas e Unificação

11. O predicado `is_set/1` recebe como parâmetro uma lista de inteiros e retorna verdadeiro se todos os elementos são únicos. Descreva o comportamento da consulta

```
?- is_set(1, 2, 3).
```

Solução:

O predicado foi declarado como `is_set/1`: ele recebe **um único** argumento (uma lista).

A consulta `?- is_set(1, 2, 3).` tenta chamar `is_set/3`, que não está definido. Portanto, o Prolog reporta um **erro** de predicado inexistente (erro de aridade, “Unknown procedure”). Para funcionar, a consulta deveria passar uma lista, por exemplo `?- is_set([1, 2, 3]).`

12. Determine, sem executar, o resultado da consulta:

```
?- X = Y, Y \= Z, Z = W, X = a, W = b.
```

Execute esta consulta no *listener* Prolog e compare com sua expectativa. Caso sua expectativa seja diferente do resultado, como a consulta deve ser modificada para obter o retorno esperado?

Solução:

A consulta **falha** (retorna `false`). Analisando passo a passo:

- `X = Y`: X e Y passam a compartilhar a mesma variável (ambos livres).
- `Y \= Z`: como Y e Z são livres, elas **são** unificáveis; logo a desigualdade `\=` falha e a conjunção termina.

Como a segunda meta já falha, o restante (`Z = W, X = a, W = b`) nunca chega a ser avaliado.

Para obter o sucesso esperado, basta instanciar Y e Z com valores distintos **antes** de testar a desigualdade, reordenando as metas. Por exemplo, a consulta `?- X = Y, Z = W, X = a, W = b, Y \= Z.` retorna **verdadeiro**, com $X = Y = a$ e $Z = W = b$.

13. O código abaixo implementa o predicado `same_parity/2`, que retorna verdadeiro se X e Y são ambos pares ou ambos ímpares.

```
1 same_parity(X, Y) :-  
2   mod(X, 2) = mod(Y, 2).
```

Contudo, a consulta

```
?- same_parity(2, 4).
```

retorna falso. Explique o que há de incorreto na implementação e proponha uma correção.

Solução:

O problema é o uso do operador de **unificação** = no lugar de uma avaliação aritmética. A expressão `mod(X, 2)` é um termo composto, não é avaliada: `mod(X, 2) = mod(Y, 2)` apenas unifica os dois termos, forçando $X = Y$. Por isso, `?- same_parity(2, 4).` tenta unificar 2 com 4 e retorna **false**.

A correção é usar o operador de comparação aritmética `==`, que avalia ambos os lados:

```
same_parity(X, Y) :-  
  X mod 2 == Y mod 2.
```